

AI Operations (AIOps)

Module 3 Student Handouts

Quick-reference sheets for Lectures 1 & 2 — keep these next to you during labs

Handout 1 — Docker & Containerization for AI

Big idea: a Conda environment.yml pins Python packages, but NOT the OS, system libraries, or GPU drivers. Containers package the ENTIRE runtime — that's the gap Module 1's protocol left open.

Images vs. Containers vs. Registries

Term	Meaning
Image	Read-only, layered template built from a Dockerfile
Container	A running (or stopped) instance of an image
Registry	Where images are stored/shared (Docker Hub, private registry)

Dockerfile Basics & Layer Caching

- Each instruction (FROM, COPY, RUN, ...) produces one cached layer.
- Put things that change RARELY (dependencies) before things that change OFTEN (app code) — maximizes cache reuse.

Multi-Stage Builds

- Stage 1 ("builder"): install/compile everything needed to build.
- Stage 2 (runtime): COPY --from=builder only the finished artifacts — no compilers, no dev headers.
- Smaller final image = faster pulls, smaller attack surface, faster cold starts (matters again in Lecture 2's KServe).

GPU Passthrough

- Base image must use a CUDA version COMPATIBLE with (not necessarily identical to) the host's driver.
- docker run --gpus all <image> — grants access to all host GPUs.
- docker run --gpus "'device=0'" <image> — restrict to one specific GPU (useful on a shared machine).

Lab 1 Command / Code Cheat Sheet

```
FROM python:3.10-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
CMD ["python", "train.py"]

$ docker build -t my-image .
```

```
$ docker images my-image
```

```
$ docker run --gpus all my-image python3 -c "import torch; print(torch.cuda.is_available())"
```

Handout 2 — Kubernetes for AI/ML Workloads & Serverless Inference (KServe)

Big idea: Docker packages ONE reproducible unit. Kubernetes decides WHERE hundreds of those units run, restarts what fails, and shares cluster resources fairly.

Core Kubernetes Objects

Object	AI/ML Relevance
Pod	One training/inference container (usually)
Deployment	Long-running service, e.g. an inference endpoint
Job	Run-to-completion, e.g. a training run — NOT a Deployment!
Service	Stable network endpoint for a changing set of Pods
Node	A physical/virtual machine that runs Pods

Jobs vs. Deployments — Don't Mix These Up

- Training run → Job (restartPolicy: Never or OnFailure).
- Inference service → Deployment (runs forever, scales with traffic).
- Deploying a training script AS a Deployment = it gets endlessly restarted the moment it finishes successfully.

GPU Scheduling

```
resources:
requests:
  nvidia.com/gpu: 1
limits:
  nvidia.com/gpu: 1 # request MUST equal limit -- no GPU overcommit
```

Infrastructure as Code

- Manifests are DECLARATIVE YAML, version-controlled in Git — describe desired state, K8s reconciles reality toward it.
- Same reproducibility principle as Module 1, applied to infrastructure instead of models.

KServe: Serverless Inference

Concept	Meaning
InferenceService	KServe's CRD wrapping Deployment+Service+autoscaling for a model
Scale-to-zero	Idle service releases ALL replicas; first request after idling pays a "cold start"
Concurrency-based autoscaling	Scales on in-flight requests per replica, not raw CPU
Canary rollout	Split traffic by % between two model revisions before a full rollout

Lab 2 Command / Code Cheat Sheet

```
# Validate manifests without touching a real cluster:
$ kubectl apply --dry-run=client -f training-job.yaml -o yaml

$ kubectl apply -f training-job.yaml
$ kubectl get jobs
$ kubectl logs job/model-training-job

$ kubectl apply -f inference-service.yaml
$ kubectl get inferencservice image-classifier

# Canary: shift 10% of traffic to the new revision
$ kubectl patch inferencservice image-classifier --type merge \
  -p '{"spec":{"predictor":{"canaryTrafficPercent": 10}}}'
```

End-of-Module Checklist

1. A working multi-stage Dockerfile with a documented size reduction vs. a naive build.
2. A correct explanation of when to use a Job vs. a Deployment for a given AI workload.
3. A validated (or applied) Job manifest with GPU resource requests/limits.
4. A validated (or applied) KServe InferenceService manifest, including a canary traffic split.